CRUD OPERATION

```
package main

import (
      "encoding/json"
      "fmt"
      "net/http"
      "strconv"
)
type User struct{
      Id int `json:"id"`
      Name string `json:"name"`
      Age int `json:"age"`
}

var users=[]User{
      {Id:1,Name: "aditya",Age: 23},
      {Id:2,Name: "naveen",Age: 23},
      {Id:3,Name: "vaibhav",Age: 21},

}

func user(w http.ResponseWriter, r *http.Request){
      switch r.Method{
      case http.MethodGet:
            getUser(w,r)

      case http.MethodPost:
            createUser(w,r)

      case http.MethodPut:
            updateUser(w,r)

      case http.MethodDelete:
            deleteUser(w,r)

    default:
            http.Error(w,"Method not allowed",http.StatusMethodNotAllowed)

      }
}

/*
func getUser(w http.ResponseWriter,r *http.Request){
w.Header().Set("Content-Type","application/json")

json.NewEncoder(w).Encode(users)

}*/
func getUser(w http.ResponseWriter,r *http.Request ){
  idString:=r.URL.Query().Get("id")

  id,err:=strconv.Atoi(idString)
  if err!=nil{
      http.Error(w,"Invalid ID",http.StatusBadRequest)
      return
  }

  for _,user :=range users{
      if user.Id==id{
```

```
            w.Header().Set("Content-Type","application/json")
          json.NewEncoder(w).Encode(user)
          return
        }
    }
http.Error(w,"User not found",http.StatusNotFound)

}

func createUser(w http.ResponseWriter,r *http.Request){
      var user User

      err:=json.NewDecoder(r.Body).Decode(&user)

      if err!=nil{
            http.Error(w,"Invalid json",http.StatusBadRequest)
            return
      }

      user.Id=len(users)+1

      users=append(users, user)

      w.Header().Set("Content-Type","Application/json")
      w.WriteHeader(http.StatusCreated)
      json.NewEncoder(w).Encode(user)



}

func updateUser(w http.ResponseWriter,r *http.Request){
      idString:=r.URL.Query().Get("id")
      id,err:=strconv.Atoi(idString)
      if err!=nil{
            http.Error(w,"Invalid",http.StatusBadRequest)
            return
      }

      var user User
      json.NewDecoder(r.Body).Decode(&user)
     for _,it:=range users{
            if it.Id==id{
                  it.Name=user.Name
                  it.Age=user.Age

      w.Header().Set("Content-Type","Application/json")
      w.WriteHeader(http.StatusCreated)
      json.NewEncoder(w).Encode(it)


      return

            }
      }

      http.Error(w,"User not found",http.StatusNotFound)
}


func deleteUser(w http.ResponseWriter,r *http.Request){
      idString:=r.URL.Query().Get("id")
      id,err:=strconv.Atoi(idString)
      if err!=nil{
```

```
			http.Error(w,"Invalid",http.StatusBadRequest)
			return
		}

		for i,user:=range users{
			if user.Id==id{
				users=append(users[:i],users[i+1:]...)
				w.WriteHeader(http.StatusNoContent)
				return
			}
		}

		http.Error(w,"User not found",http.StatusNotFound)
	}
	func main(){
		mux:=http.NewServeMux()

		mux.HandleFunc("/user",user)
		server:=&http.Server{
			Addr: ":8000",
			Handler: mux,
		}
		fmt.Println("Server is running 8000")

		err:=server.ListenAndServe()
		if err!=nil{
			fmt.Println(err)
		}
	}
```